CS247 Assignment 3 Marching Cubes (3D iso-surface)

How to run

```
cd Assignment_3
cmake -S . -B build
cmake --build build -j
cd build && ./assignment
```

Press 1 for lobster (fast) or 2 for head (~590k triangles, takes a couple of seconds to extract). The surface shows in the second window.

Key	Action
1 / 2	load lobster / head
I / O	iso +0.01 / +0.1 (re-runs marching cubes)
K / L	iso −0.01 / −0.1
W / S / A	slice / axis (2D window only)
left-drag	rotate, Ctrl+drag zoom, Home reset
Esc	quit

Implementation

Marching cubes builds a triangle iso-surface with normals from the gradient, and I do Gouraud shading. The 2D contour from A2 still works in the first window. Geometry is only rebuilt when the iso value changes; the render loop just redraws the existing buffer.

Marching cubes. Walk every cube cell, sample the 8 corners, build an 8-bit case index:

```
for (int c = 0; c < 8; ++c)
    if (vC[c] > iso + eps) cubeIdx |= (1 << c);
```

`edgeTable3D[cubeIdx]` gives the 12-bit edge mask, `triangleTable[cubeIdx]` gives up to 5 triangles. `cornerOffset` / `edgeCorners` follow the Lorensen Cline numbering so the supplied tables match.

Vertex positions. Each crossed edge is interpolated `t = (iso-vA)/(vB-vA)`, `pos = (1-t)·pA + t·pB`, in voxel-centre NDC.

Normals. I precompute a gradient field once at load time with central differences (forward/backward at the boundaries), so an iso change only pays for the marching cubes pass:

```
gx = 0.5 * (V(x+1,y,z) - V(x-1,y,z));   // central, interior
```

On each edge I lerp the two endpoint gradients by the same `t`, normalise, and flip the sign so the normal points outward.

Gouraud shading. Lighting is per-vertex in the vertex shader (Lambertian + ambient); the fragment shader just passes the interpolated colour through, so the smooth look is the rasteriser interpolating vertex colours:

```
mat3 normalMat = transpose(inverse(mat3(model)));
vec3 N = normalize(normalMat * vertexNorm);
vec3 L = normalize(lightPos - worldPos.xyz);
fragColor = vertexColor * (0.18 + max(dot(N, L), 0.0));
```

A `useLighting` uniform lets the same shader also draw the unlit wireframe box.

The surface is an interleaved (pos, normal) `GL_STREAM_DRAW` VBO, re-uploaded on iso change and drawn with `glDrawArrays(GL_TRIANGLES, ...)`. The VBO lives in the 3D window's context, so I switch context before uploading.

Results

See `demo.mov`. The terminal prints `marching cubes: N triangles` on every rebuild. Phong / movable-light bonuses not done.